1. Introducción

Arreglo de estructuras en C

Un **arreglo de estructuras** permite almacenar **varios objetos del mismo tipo**.

Ejemplo: almacenar varios sensores.

Definición de la estructura

```
struct Sensor {
    int id;
    float temperatura;
};
```

Declaración de un arreglo de estructuras

```
struct Sensor sensores[3];
```

Esto crea **3 sensores diferentes en memoria**.

Acceso a los elementos

```
sensores[0].id = 1;
sensores[0].temperatura = 25.3;

sensores[1].id = 2;
sensores[1].temperatura = 26.7;
```

Ejemplo completo

```
#include <stdio.h>

struct Sensor {
   int id;
   float temperatura;
};

int main() {

   struct Sensor sensores[2];

   sensores[0].id = 1;
   sensores[0].temperatura = 24.5;

   sensores[1].id = 2;
   sensores[1].temperatura = 26.1;

   printf("Sensor %d Temp: %.2f\n", sensores[0].id,
sensores[0].temperatura);
   printf("Sensor %d Temp: %.2f\n", sensores[1].id,
sensores[1].temperatura);

}
```

Aplicación en microcontroladores

Esto se usa para manejar:

- múltiples sensores
- buffers de datos
- arreglos de LEDs
- múltiples motores

1. Introducción

typedef struct

Uso de typedef struct

En C, normalmente debemos escribir:

struct Sensor sensor1;

Pero esto puede volverse repetitivo.

Solución: typedef

Permite **crear un alias del tipo de dato**.

Definición:

```
typedef struct {

    int id;
    float temperatura;

} Sensor;
```

Ahora podemos declarar variables así:

```
Sensor s1;
Sensor s2;
```

1. Introducción

Ejemplo completo

```
#include <stdio.h>

typedef struct {

    int id;
    float temperatura;

} Sensor;

int main() {

    Sensor s1;

    s1.id = 5;
    s1.temperatura = 27.8;

    printf("Sensor %d Temp: %.2f\n", s1.id, s1.temperatura);

}
```

Ventaja: El código es más **limpio y legible**.

Esto es muy usado en **programación embebida**.

1. Introducción

Ejemplo con microcontrolador STM32 (typedef struct)

Control de LEDs usando typedef struct

La tarjeta **STM32H755ZI Nucleo-144** tiene **3 LEDs de usuario**:

•**LD1 (verde)** → PB0 o PA5
•**LD2 (amarillo)** → PE1
•**LD3 (rojo)** → PB14

Podemos representar cada LED con una **estructura**.

```
typedef struct {

    GPIO_TypeDef *puerto;
    uint16_t pin;

} LED;
```

Esta estructura almacena:

- el **puerto GPIO**
- el **pin del LED**

Declaración de los LEDs

```
LED leds[3] = {

    {GPIOB, GPIO_PIN_0},   // LD1
    {GPIOE, GPIO_PIN_1},   // LD2
    {GPIOB, GPIO_PIN_14}   // LD3

};
```

Aquí usamos **un arreglo de estructuras**.